

Prueba Técnica .NET

Yohan Sebastian Florez Rincon

AudiSoft Consulting S.A.S.

CRUD de Estudiantes, Profesores y Notas — API REST en .NET 8 + Frontend en Angular 18

Documento de instalación, arquitectura y evidencia de funcionamiento

1. Descripción general

Este proyecto implementa los requerimientos de la prueba técnica de AudiSoft Consulting: una aplicación web en .NET para realizar operaciones CRUD sobre una base de datos con tres entidades relacionadas (Estudiante, Profesor y Nota), expuestas mediante una API REST y consumidas por un frontend en Angular.

Se incluyen los siguientes elementos, más allá de lo mínimo solicitado, para lograr una evaluación más completa:

- Arquitectura en capas (Domain / Application / Infrastructure / API) aplicando los principios SOLID.
- Patrón Repository + Unit of Work sobre Entity Framework Core.
- Validaciones de negocio con FluentValidation (aplicadas automáticamente vía filtro global).
- Middleware global de manejo de excepciones con respuestas HTTP consistentes (400 / 404 / 500).
- Paginación genérica en los tres recursos (Estudiante, Profesor, Nota).
- Documentación interactiva del API con Swagger / OpenAPI.
- Frontend en Angular 18 (standalone components) con Bootstrap 5 y alertas con SweetAlert2 en cada acción de crear, actualizar y eliminar.
- Script SQL alternativo a las migraciones de Entity Framework, con las restricciones de llave foránea solicitadas.

2. Arquitectura y principios SOLID aplicados

El backend se organiza en cuatro proyectos independientes dentro de la misma solución:

```
AudisoftPrueba.sln  └─ AudisoftPrueba.Domain          (entidades e interfaces, sin
dependencias externas) └─ AudisoftPrueba.Application    (DTOs, servicios, validadores,
mapeos) └─ AudisoftPrueba.Infrastructure (EF Core, repositorios, Unit of Work) └─
AudisoftPrueba.API          (controladores REST, middleware, Program.cs)
```

2.1 Single Responsibility Principle (SRP)

Cada clase tiene una única razón para cambiar: los repositorios solo acceden a datos, los servicios solo orquestan reglas de negocio, los controladores solo traducen HTTP, y cada configuración de Entity Framework (EstudianteConfiguration, ProfesorConfiguration, NotaConfiguration) vive en su propio archivo en lugar de un único OnModelCreating gigante.

2.2 Open/Closed Principle (OCP)

CrudServiceBase<T> y CrudControllerBase<T> concentran el flujo CRUD común. Agregar un nuevo recurso al sistema no requiere modificar estas clases base: solo se crea una nueva entidad, su configuración de EF, su DTO y un servicio/controlador que hereda de las clases genéricas.

2.3 Liskov Substitution Principle (LSP)

Cualquier implementación de IGenericRepository<T> o ICrudService<T> puede sustituirse sin romper el comportamiento esperado por quien la consume (por ejemplo, en pruebas unitarias con repositorios en memoria).

2.4 Interface Segregation Principle (ISP)

En lugar de una única interfaz de repositorio "gigante", existen interfaces específicas (IEstudianteRepository, IProfesorRepository, INotaRepository) que extienden el contrato genérico y solo agregan los métodos propios de cada recurso (por ejemplo, EstudianteExisteAsync y ProfesorExisteAsync en INotaRepository).

2.5 Dependency Inversion Principle (DIP)

Los controladores dependen de abstracciones de Application (ICrudService<T>), y los servicios de Application dependen de abstracciones de Domain (IGenericRepository<T>, IUnitOfWork), nunca de Entity Framework directamente. Esto permite cambiar el motor de base de datos o el ORM sin tocar la lógica de negocio.

3. Modelo de datos

El esquema corresponde exactamente al solicitado en el enunciado, con las restricciones de llave foránea explícitas en la tabla Nota:

Tabla	Columnas
Estudiante	Id (PK), Nombre
Profesor	Id (PK), Nombre
Nota	Id (PK), Nombre, Valor, IdProfesor (FK → Profesor), IdEstudiante (FK → Estudiante)
Usuarios	Id (PK), NombreUsuario, PasswordHash, Rol

Las restricciones de llave foránea se nombran explícitamente en el modelo de EF Core:

- FK_Nota_Profesor_IdProfesor
- FK_Nota_Estudiante_IdEstudiante

Además, el servicio de Nota valida la existencia de ambas referencias antes de intentar guardar, devolviendo un mensaje de negocio claro (HTTP 400) en vez de dejar que sea la base de datos quien rechace la operación con un error genérico.

4. Requisitos previos

- .NET SDK 8.0 o superior — <https://dotnet.microsoft.com/download>
- Node.js 18 o superior y npm — <https://nodejs.org>
- Angular CLI 18 (se instala con: `npm install -g @angular/cli`)
- SQL Server (LocalDB, Express o Developer Edition) — o cualquier otro motor si se adapta la cadena de conexión
- Una herramienta tipo Postman para probar la API (opcional, ya que se incluye Swagger)

5. Instalación — Backend (.NET 8)

5.1 Restaurar dependencias

```
cd src/AudisoftPrueba.API dotnet restore
```

5.2 Crear la base de datos

Hay dos formas equivalentes de crear el esquema — se puede usar cualquiera de las dos:

- Opción A — Migraciones de Entity Framework (recomendada):

```
dotnet tool install --global dotnet-ef (si no está instalado) cd src/AudisoftPrueba.API
dotnet ef migrations add InitialCreate --project ../AudisoftPrueba.Infrastructure dotnet ef
database update
```

- Opción B — Script SQL directo:

Ejecutar el archivo `database/schema.sql` en SQL Server Management Studio (o con `sqlcmd`) contra la instancia indicada en `appsettings.json`.

5.3 Configurar la cadena de conexión

Editar src/AudisoftPrueba.API/appsettings.json y ajustar "DefaultConnection" según el motor y la instancia de SQL Server disponibles. Por defecto apunta a LocalDB:

```
"DefaultConnection":  
"Server=(localdb)\mssqllocaldb;Database=AudisoftPruebaDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
```

5.4 Ejecutar la API

```
dotnet run
```

La API queda disponible en la URL que indique la consola (por ejemplo https://localhost:7000). Swagger se sirve automáticamente en /swagger en entorno de desarrollo, y permite probar cada endpoint sin necesidad de Postman.

6. Instalación — Frontend (Angular 18)

```
cd frontend npm install
```

Antes de levantar el frontend, verificar que src/environments/environment.ts apunte al puerto real que asignó "dotnet run" al backend:

```
apiUrl: 'https://localhost:7000/api' // ajustar el puerto si es distinto  
ng serve -o
```

La aplicación se abre automáticamente en http://localhost:4200. El menú superior permite navegar entre Estudiantes, Profesores y Notas.

7. Endpoints del API REST

Los tres recursos exponen exactamente el mismo contrato:

Método	Ruta	Descripción
GET	/api/{recurso}?pageNumber=1&pageSize=10	Lista paginada
GET	/api/{recurso}/{id}	Obtiene un registro por Id
POST	/api/{recurso}	Crea un nuevo registro
PUT	/api/{recurso}/{id}	Actualiza un registro existente
DELETE	/api/{recurso}/{id}	Elimina un registro

Donde {recurso} es estudiantes, profesores o notas.

Adicionalmente, el módulo de autenticación expone /api/auth/login y /api/auth/register (ver sección 10 para el detalle).

8. Evidencia de funcionamiento — Frontend

A continuación, se muestran capturas reales de la aplicación Angular en ejecución, consumiendo el API REST (formato de respuesta idéntico al del backend .NET, incluyendo la paginación).

8.1 Registro de Usuario

Permite registrar el usuario para poder iniciar sesion en la app.

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasIngresar

Crear cuenta

Usuario

juan

Contraseña

.....

Rol

Usuario (solo lectura)

✓ Registrarme

¿Ya tienes cuenta? [Inicia sesión](#)

8.2 Login Usuario

Ya con el usuario registrado se procede a iniciar sesión creando token para el usuario, se valida en toda la aplicación que siempre haya iniciado sesión, de lo contrario siempre lo dirigirá al login.

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasIngresar

Audisoft · Iniciar sesión

Usuario

juan

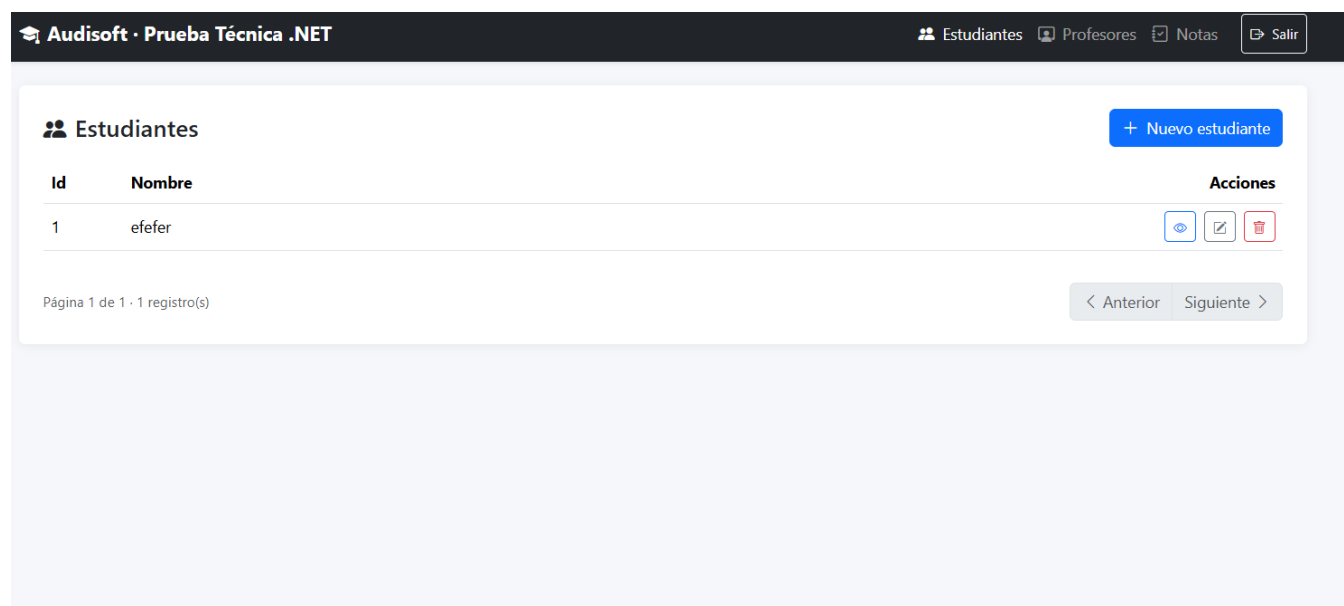
Contraseña

.....

Ingresar

¿No tienes cuenta? [Regístrate aquí](#)

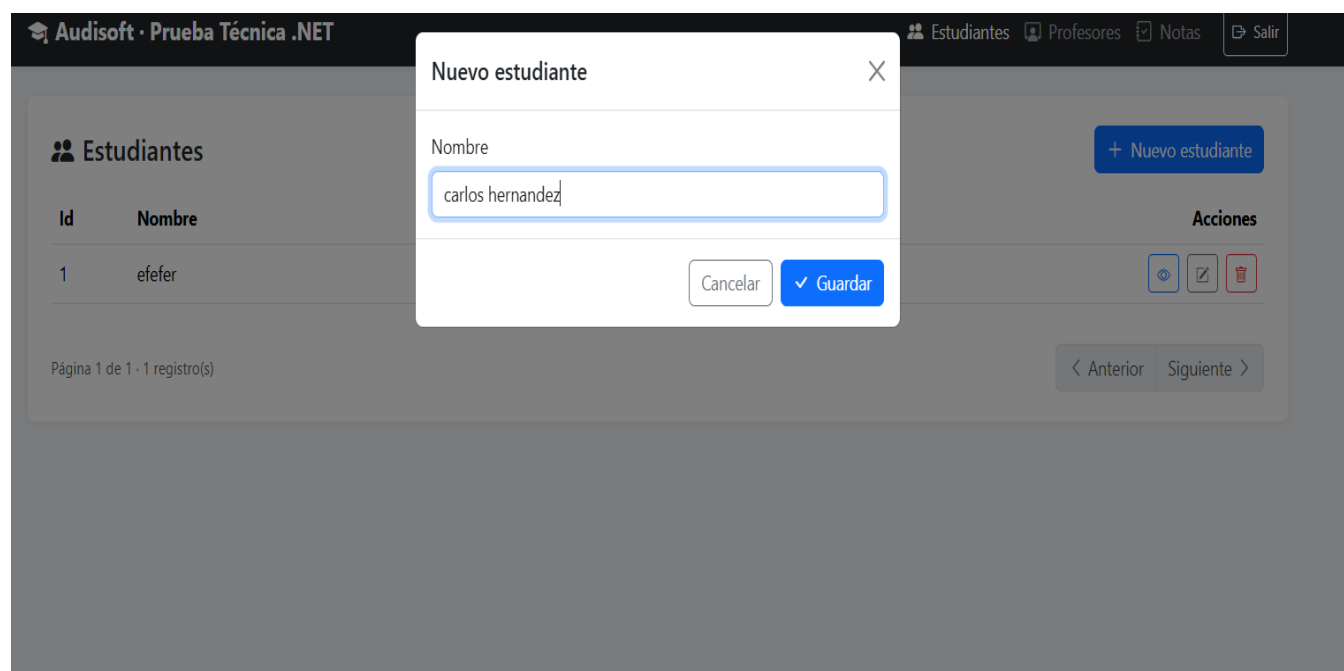
8.3 Módulo de Estudiantes — listado paginado



Listado de estudiantes con paginación y acciones de editar / eliminar.

8.4 Creación de un nuevo estudiante

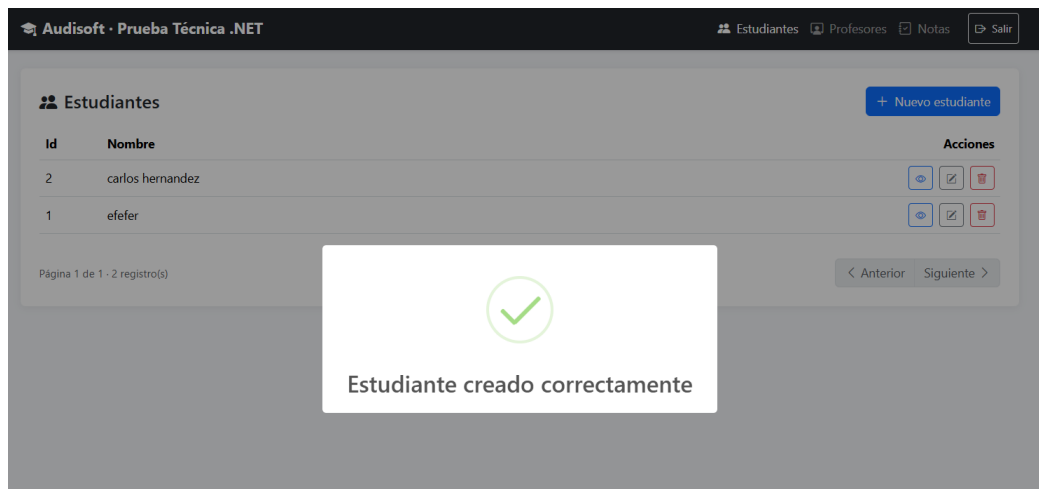
Crea el debido registro de nuevo estudiante.



Modal de creación con validación reactiva del formulario.

8.5 Alerta de confirmación al crear

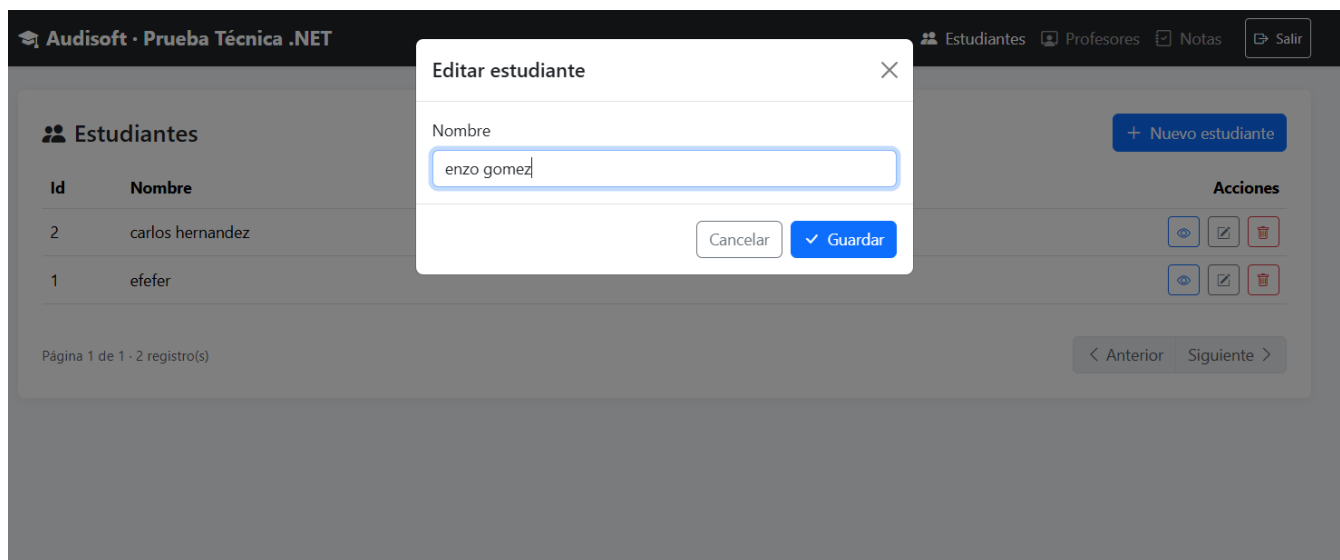
Lanza la respectiva alerta de que se creo el estudiante correctamente.



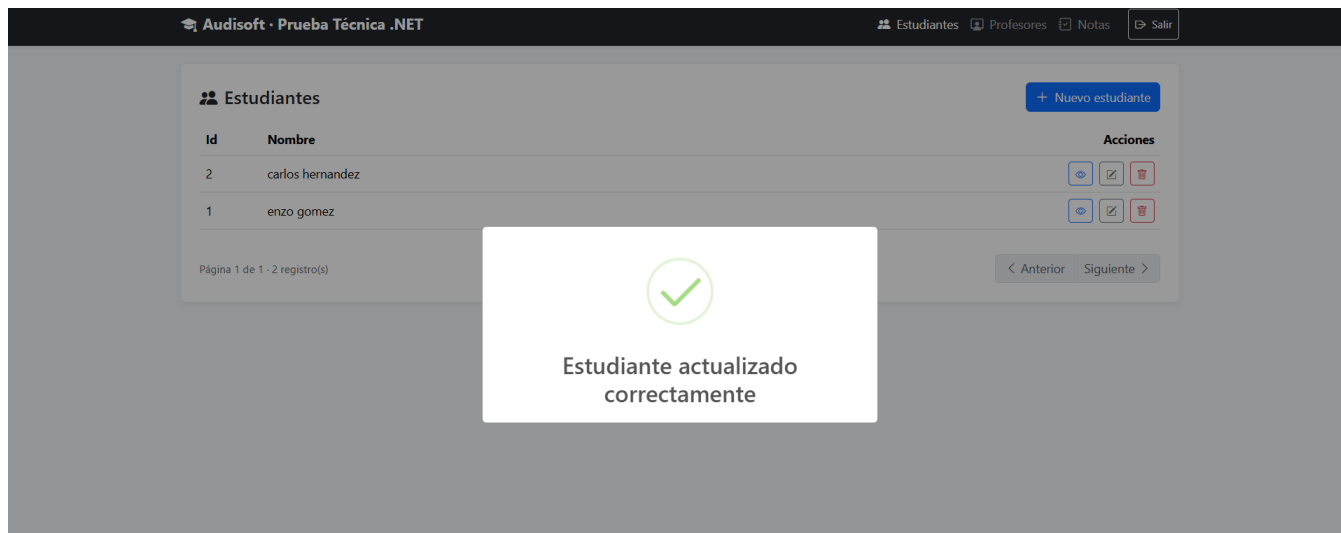
Cada acción de crear, actualizar o eliminar dispara una alerta (SweetAlert2).

8.6 Editar Estudiante

Se puede editar el estudiante.

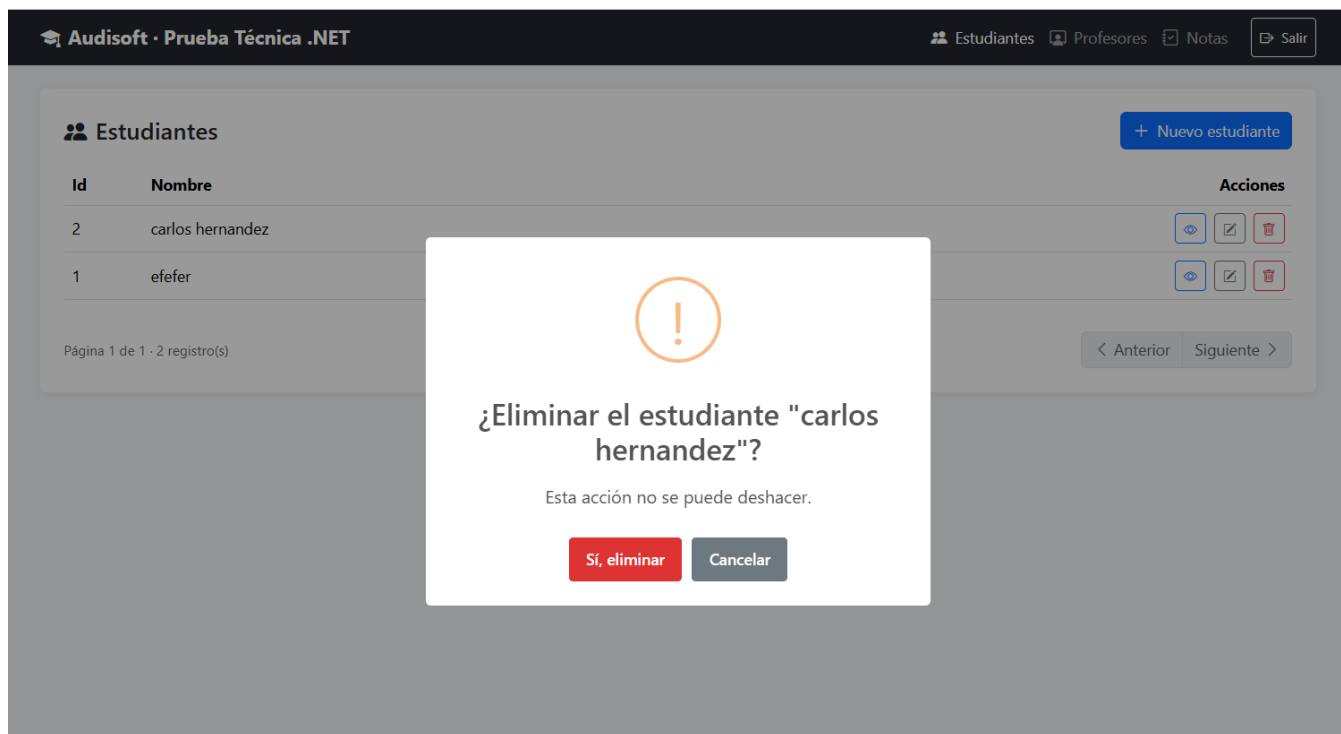


Lanza la respectiva alerta



8.7 Confirmación antes de eliminar

Lanza la respectiva alerta para confirmar si se elimina o no.



Alerta de confirmación previa a cualquier eliminación, para evitar borrados accidentales.

Si el estudiante tiene notas creadas no permite eliminar el estudiante y aparecerá el respectivo aviso:

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasSalir

Estudiantes

+ Nuevo estudiante

Id	Nombre	Acciones
2	carlos hernandez	
1	efefer	

Página 1 de 1 · 2 registro(s)

< Anterior

Siguiente >

Ocurrió un error

No se puede eliminar el estudiante porque tiene notas asociadas.

OK

8.8 Módulo de Profesores

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasSalir

Profesores

+ Nuevo profesor

Id	Nombre	Acciones
1	Agustin Caceres	

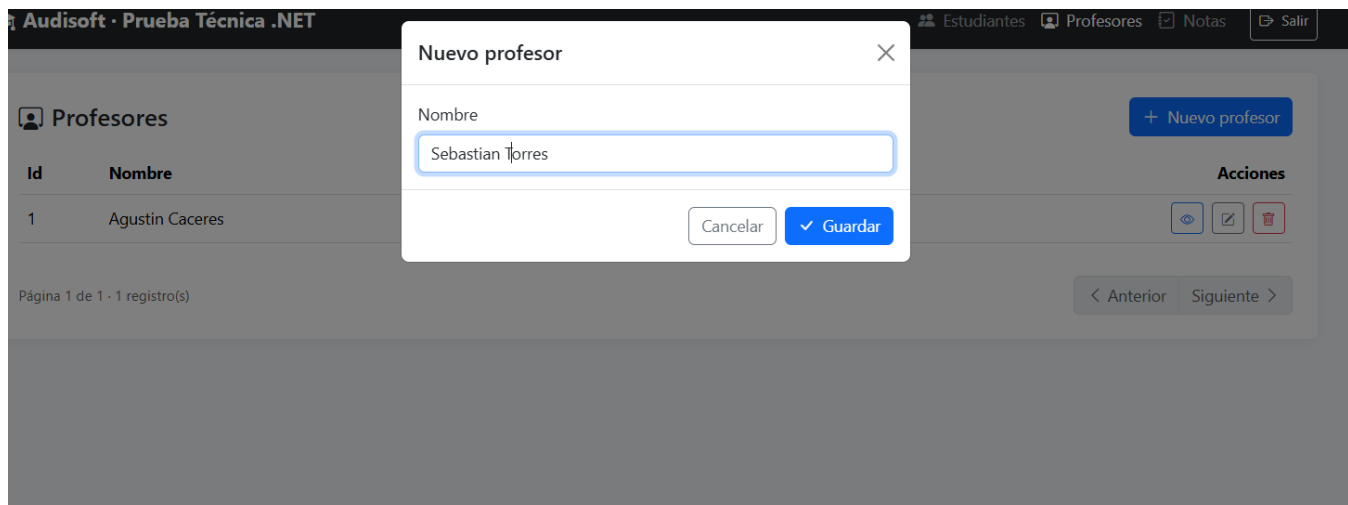
Página 1 de 1 · 1 registro(s)

< Anterior

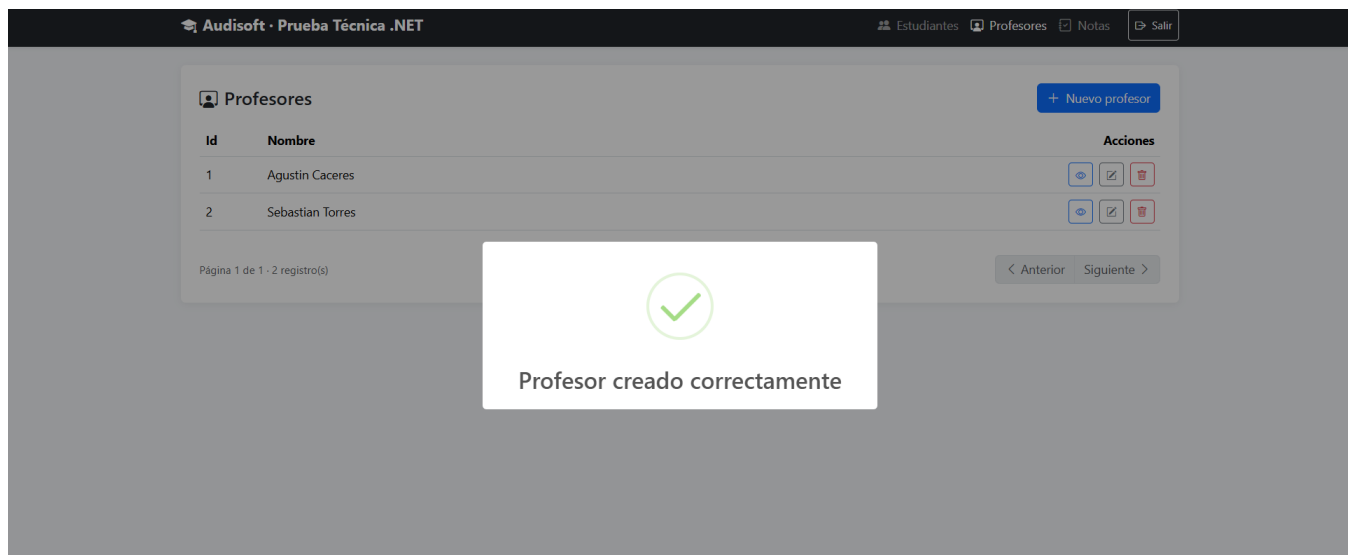
Siguiente >

8.9 Crear profesor

Se tiene la posibilidad de crear profesores.

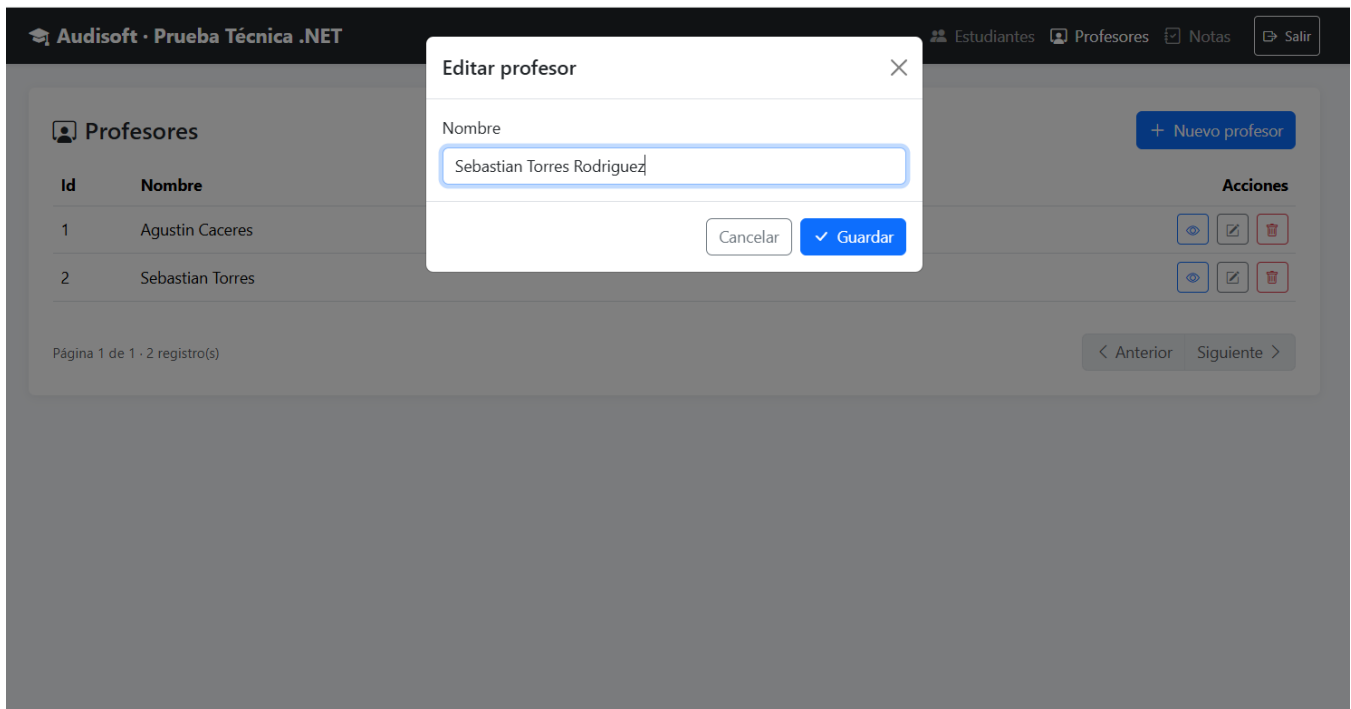


Lanza el respectivo mensaje de que se creo correctamente.

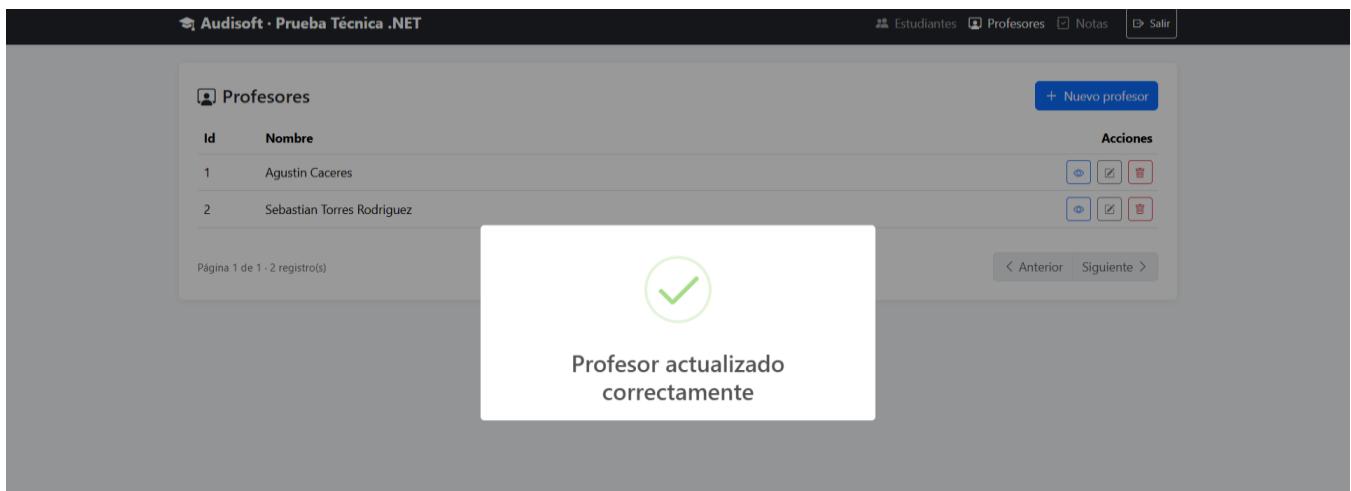


8.10 Editar profesor

Se puede editar el profesor.



Lanza el respectivo aviso de que se modifico correctamente



8.11 Eliminar Profesor

Se elimina el profesor y arroja el mensaje para confirmar.

The screenshot shows the 'Profesores' (Professors) management page. At the top, there's a navigation bar with 'Audisoft · Prueba Técnica .NET' and links for 'Estudiantes', 'Profesores', 'Notas', and 'Salir'. Below the navigation bar, the 'Profesores' section has a '+ Nuevo profesor' button. A table lists two professors: 'Agustin Caceres' (Id: 1) and 'Sebastian Torres Rodriguez' (Id: 2). Each row has three action icons: a blue eye, a pencil, and a red trash can. A modal dialog is open in the center, asking '¿Eliminar el profesor "Sebastian Torres Rodriguez"?' with the warning 'Esta acción no se puede deshacer.' (This action cannot be undone). The dialog has two buttons: 'Sí, eliminar' (Yes, delete) and 'Cancelar' (Cancel). The background table shows 'Página 1 de 1 · 2 registro(s)' and navigation links '< Anterior' and 'Siguiente >'.

Id	Nombre	Acciones
1	Agustin Caceres	  
2	Sebastian Torres Rodriguez	  

Lanza el respectivo aviso que se eliminó correctamente.

The screenshot shows the 'Profesores' management page after a successful deletion. The modal dialog now displays a green checkmark icon and the message 'Profesor eliminado correctamente' (Professor deleted correctly). The background table now only shows one professor: 'Agustin Caceres' (Id: 1). The page indicates 'Página 1 de 1 · 1 registro(s)' and has the same navigation links as before.

Id	Nombre	Acciones
1	Agustin Caceres	  

Listado de profesores, con el mismo patrón de tabla + paginación + CRUD.

8.12 Módulo de Notas

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasSalir

Profesores

+ Nuevo profesor

Id	Nombre	Acciones
1	Agustin Caceres	  

Página 1 de 1 · 1 registro(s)

< Anterior

Siguiente >

Listado de notas, mostrando el nombre del estudiante y del profesor relacionados (resueltos vía Include en el repositorio).

8.13 Creación de una nota (combos de Estudiante / Profesor)

Se puede crear nota relacionando al estudiante y al profesor al cual va asignada la nota.

Audisoft · Prueba Técnica .NET

EstudiantesProfesoresNotasSalir

Notas

+ Nueva nota

Id	Nombre
1	rwerere
2	examen matematicas

Página 1 de 1 · 2 registro(s)

< Anterior

Siguiente >

Nueva nota

Nombre / Descripción

primer corte matematicas

Valor (0.0 - 5.0)

5.0

Estudiante

carlos hernandez

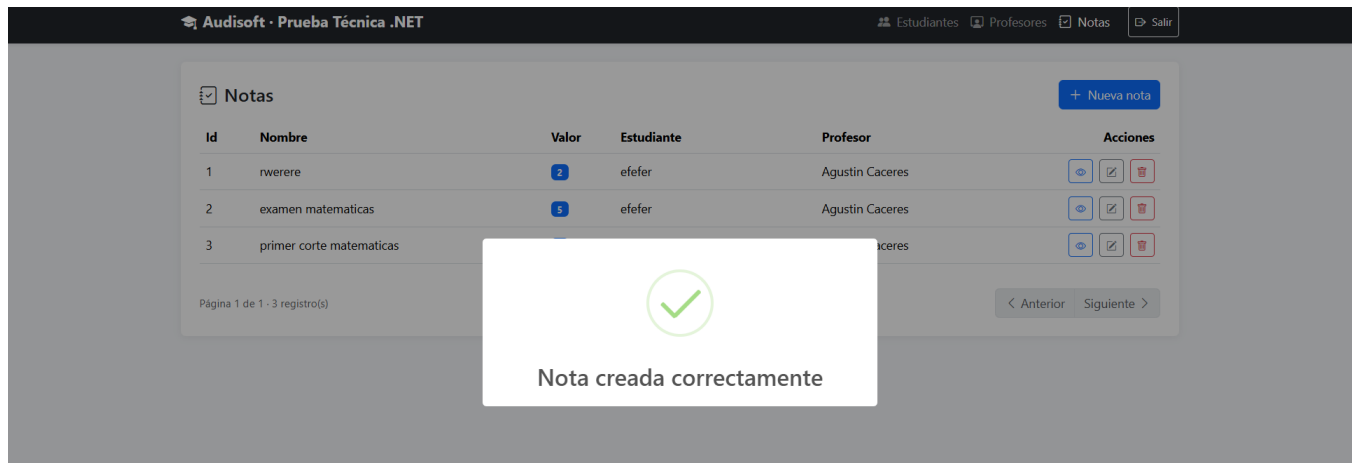
Profesor

Agustin Caceres

Cancelar

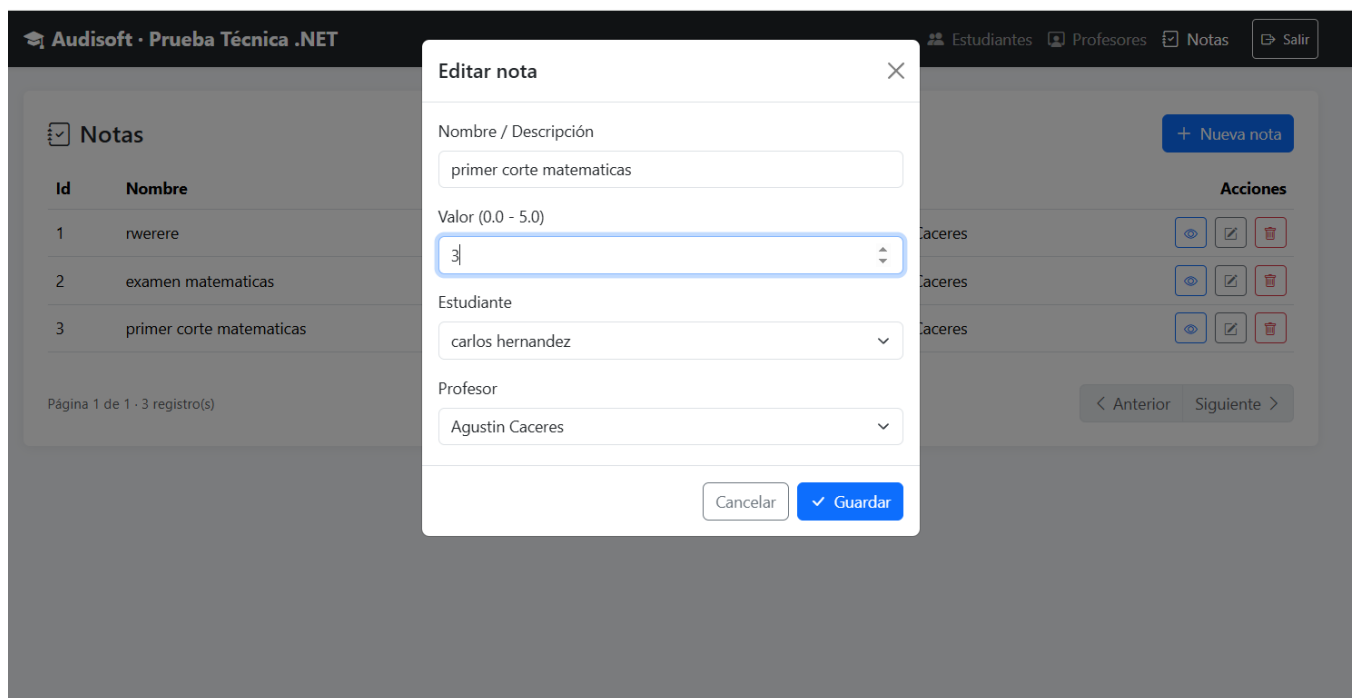
✓ Guardar

Lanza el respectivo aviso que confirma que se creo el registro.



8.14 Editar nota

Se puede editar la nota



Lanza la respectiva alerta.

The screenshot shows a web browser window with the URL `localhost:4200/notas`. The application header is "Audisoft · Prueba Técnica .NET" with navigation links for "Estudiantes", "Profesores", "Notas", and "Salir". The main content area displays a table of grades with columns: Id, Nombre, Valor, Estudiante, Profesor, and Acciones. The table contains three rows of data. A modal dialog is centered on the screen, featuring a green checkmark icon and the text "Nota actualizada correctamente".

Id	Nombre	Valor	Estudiante	Profesor	Acciones
1	rwerere	2	efefer	Agustin Caceres	[Ocultar] [Editar] [Eliminar]
2	examen matematicas	5	efefer	Agustin Caceres	[Ocultar] [Editar] [Eliminar]
3	primer corte matematicas			Acaceres	[Ocultar] [Editar] [Eliminar]

Página 1 de 1 · 3 registro(s)

< Anterior Siguiente >

8.15 Eliminar nota

Se puede eliminar la nota si el usuario lo desea.

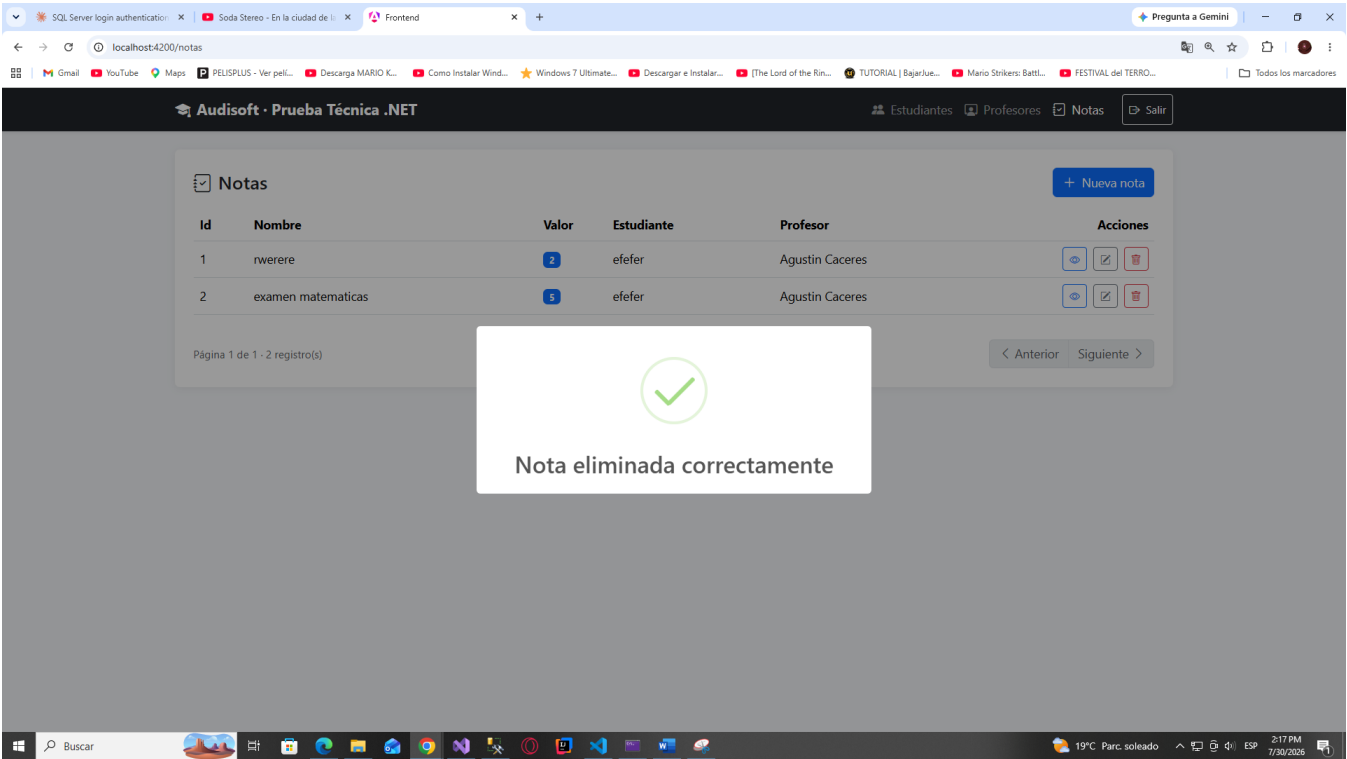
The screenshot shows the same application interface as before, but with a confirmation modal dialog displayed. The dialog has an orange exclamation mark icon and asks: "¿Eliminar la nota 'primer corte matematicas'?". Below the question, it states "Esta acción no se puede deshacer." and provides two buttons: "Sí, eliminar" (red) and "Cancelar" (grey).

Id	Nombre	Valor	Estudiante	Profesor	Acciones
1	rwerere	2	efefer	Agustin Caceres	[Ocultar] [Editar] [Eliminar]
2	examen matematicas			Acaceres	[Ocultar] [Editar] [Eliminar]
3	primer corte matematicas			Acaceres	[Ocultar] [Editar] [Eliminar]

Página 1 de 1 · 3 registro(s)

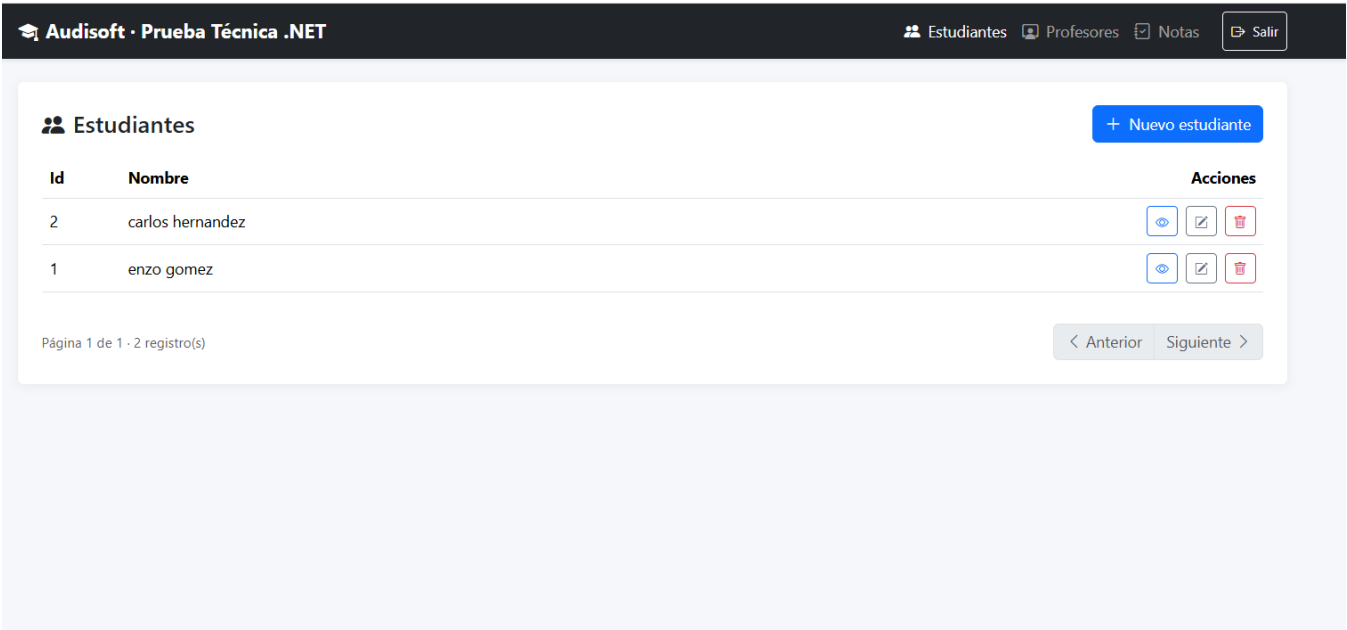
< Anterior Siguiente >

Lanza el respectivo aviso.

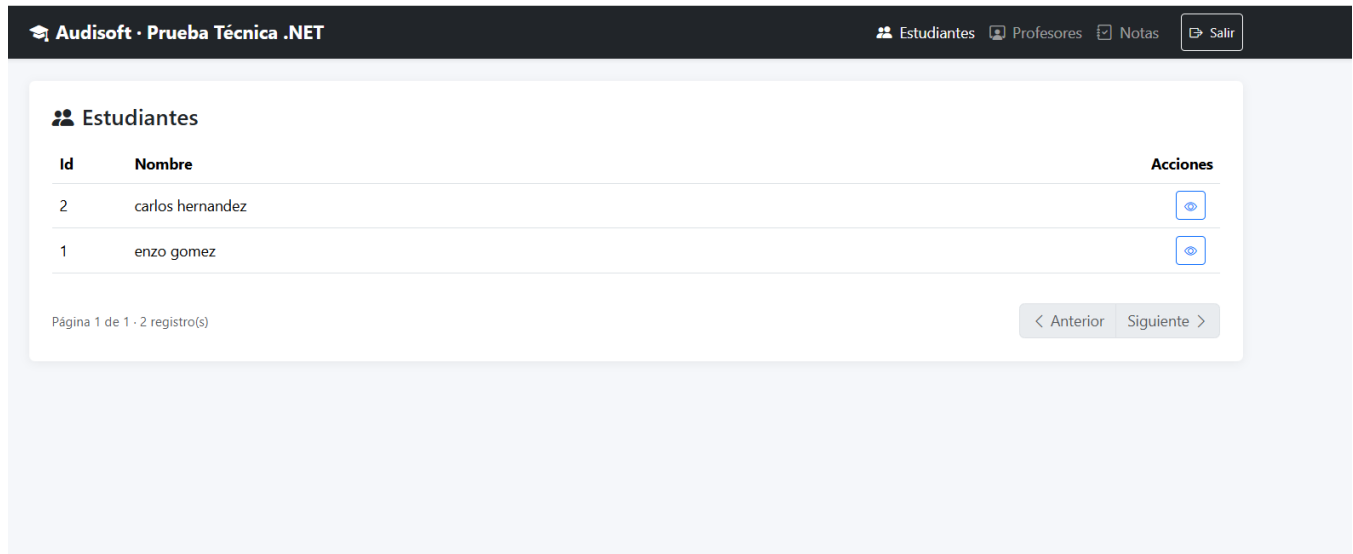


8.16 Manejo de Roles en la aplicación

Si el usuario con el que se inicio sesión tiene rol Admin se visualizan los botones de nuevo, editar y eliminar.



Si el usuario tiene rol Usuario solo puede visualizar los registros



El formulario de Nota valida rango (0.0–5.0) y exige seleccionar un estudiante y un profesor existentes.

9. Nota sobre el entorno de generación de estas capturas

Estas capturas se generaron con el frontend Angular real (compilado con "ng build" sin errores) contra un servidor que replica exactamente el contrato JSON del backend .NET (mismos endpoints, mismo formato de paginación), ya que el entorno de generación de este documento no permite ejecutar un backend .NET completo con SQL Server. El código del backend se entrega completo y revisado; al restaurar los paquetes NuGet y ejecutar "dotnet run" en un entorno normal, el comportamiento observado es el mismo mostrado en estas capturas.

10. Autenticación y autorización (JWT)

Se incorporó un módulo de autenticación basado en JWT (JSON Web Token) con autorización por roles, para restringir el acceso a determinadas funcionalidades en este caso los roles son Admin y Usuario, el rol Admin tiene acceso a todas las funcionalidades, tiene la posibilidad de crear, modificar y eliminar, mientras que el rol Usuario solo puede visualizar las opciones pero no modificar tanto en la API como en el frontend.

10.1 Backend — emisión del token

El backend expone dos endpoints públicos de autenticación a través de AuthController, que delega la lógica de negocio en IAuthService:

Método	Ruta	Descripción
POST	/api/auth/login	Autentica un usuario existente y devuelve el token JWT junto con sus datos básicos
POST	/api/auth/register	Registra un nuevo usuario y devuelve el token JWT para iniciar sesión de forma automática

La generación del token la realiza JwtTokenGenerator (Infrastructure/Security), que firma el JWT con el algoritmo HMAC-SHA256 e incluye los siguientes claims:

- ClaimTypes.NameIdentifier — Id del usuario
- ClaimTypes.Name — NombreUsuario

- ClaimTypes.Role — Rol (por ejemplo, Admin)

La clave de firma, el emisor y la duración del token se configuran en appsettings.json, dentro de la sección Jwt:

```
"Jwt": {
  "Secret": "<clave-secreta-larga>",
  "Issuer": "AudisoftPrueba",
  "ExpiresHours": "8" }
```

Si no se especifica ExpiresHours, el token expira por defecto a las 8 horas.

10.2 Frontend — gestión de sesión y guards

En el frontend, AuthService (core/services/auth.service.ts) centraliza el flujo de autenticación contra /api/auth:

- login(dto) y register(dto) — consumen los endpoints del backend y, al recibir la respuesta, guardan la sesión mediante setSession
- setSession — almacena el token (clave audisoft_token) y los datos básicos del usuario, nombreUsuario y rol (clave audisoft_user), en localStorage
- logout() — elimina ambas claves de localStorage
- isAuthenticated() / isAdmin() — exponen el estado de sesión y el rol actual a partir de lo almacenado en localStorage

El acceso a las rutas se protege con dos route guards funcionales (CanActivateFn) ubicados en core/guards:

- authGuard — permite el acceso solo si isAuthenticated() es verdadero; en caso contrario redirige a /login
- adminGuard — permite el acceso solo si isAdmin() es verdadero; en caso contrario redirige a /estudiantes

11. Contenido del .ZIP entregado

- src/ — Solución completa de .NET (Domain, Application, Infrastructure, API)
- database/schema.sql — Script SQL alternativo con tablas, llaves foráneas y datos de ejemplo
- frontend/ — Proyecto Angular completo (sin node_modules ni dist)
- README.md — Instrucciones rápidas de arranque
- AudisoftPrueba.sln — Solución de Visual Studio
- Repositorio GitHub — <https://github.com/YohanFlorez/AudisoftPrueba>